

Assignment -7

1. Class polygon contains data member width and height and public method set_value() to assign values to width and height. Class Rectangle and Triangle are inherited from polygon class. Both the classes contain public method calculate_area() to calculate the area of Rectangle and Triangle. Use base class pointer to access the derived class object and show the area calculated.

Code:-

```
[*] Untitled1
1 //Assignment- 7
2 //Question -1
3 #include <iostream>
4 using namespace std;
5 class Polygon {
6 protected:
7     int width, height;
8 public:
9     void set_value(int w, int h) {
10         width = w;
11         height = h;
12     }
13     virtual void calculate_area() = 0; // pure virtual
14 };
15 class Rectangle : public Polygon {
16 public:
17     void calculate_area() {
18         cout << "Rectangle Area: " << width * height << endl;
19     }
20 };
21 class Triangle : public Polygon {
22 public:
23     void calculate_area() {
24         cout << "Triangle Area: " << (width * height) / 2 << endl;
25     }
26 };
27 int main() {
28     Polygon *p;
29     Rectangle r;
30     Triangle t;
31     r.set_value(10, 5);
32     t.set_value(10, 5);
33 }
34
35 int main() {
36     Polygon *p;
37     Rectangle r;
38     Triangle t;
39     r.set_value(10, 5);
40     t.set_value(10, 5);
41     p = &r;
42     p->calculate_area();
43     p = &t;
44     p->calculate_area();
45 }
```

Compiler Resources Compile Log Debug Find Results

Line: 17 Col: 28 Sel: 0 Lines: 37 Length: 766 Insert

Output:-

```
Rectangle Area: 50
Triangle Area: 25
```

```
-----
Process exited after 0.2896 seconds with return value 0
Press any key to continue . . .
```

2. Write a program to create a class shape with functions area and display to find area and display the name of the shape and other essential component of the class. Create derived classes circle, rectangle and triangle each having overridden functions area and display. Write a program to find and display the area of circle, rectangle and triangle.

Code:-

```
ques1.cpp  [*] Untitled2
1 //Assignment- 7
2 //Question -2
3 #include <iostream>
4 using namespace std;
5 class Shape {
6 public:
7     virtual void area() {}
8     virtual void display() {}
9 };
10 class Circle : public Shape {
11     float r;
12 public:
13     Circle(float x) { r = x; }
14     void area() {
15         cout << "Circle Area: " << 3.14 * r * r << endl;
16     }
17     void display() {
18         cout << "Circle\n";
19     }
20 };
21 class Rectangle : public Shape {
22     int l, b;
23 public:
24     Rectangle(int x, int y) { l = x; b = y; }
25     void area() {
26         cout << "Rectangle Area: " << l * b << endl;
27     }
28     void display() {
29         cout << "Rectangle\n";
30     }
31 };
32 class Triangle : public Shape {
33     int b, h;
34 public:
35     Triangle(int x, int y) { b = x; h = y; }
36     void area() {
37         cout << "Triangle Area: " << 0.5 * b * h << endl;
38     }
39     void display() {
40         cout << "Triangle\n";
41     }
42 };
43 int main() {
44     Shape *s;
45
46     Circle c(5);
47     Rectangle r(4,6);
48     Triangle t(3,7);
49
50     s = &c; s->display(); s->area();
51     s = &r; s->display(); s->area();
52     s = &t; s->display(); s->area();
53 }
```

Output:-

```
C:\Users\asus\Desktop\oops\  X  +  v

Circle
Circle Area: 78.5
Rectangle
Rectangle Area: 24
Triangle
Triangle Area: 10.5

-----
Process exited after 0.3103 seconds with return value 0
Press any key to continue . . .
```

3. Write a C++ program to compute area of right angle triangle, equilateral triangle, isosceles triangle using function overloading concept.

Code:-

```
1 //Assignment- 7
2 //Question -3
3 #include <iostream>
4 #include <cmath>
5 using namespace std;
6 float area(float b, float h) { return 0.5 * b * h; } // right triangle
7 float area(float a) { return (1.732/4) * a * a; } // equilateral
8 float area(float a, float b, float c) { // isosceles
9     float s = (a+b+c)/2;
10    return sqrt(s*(s-a)*(s-b)*(s-c));
11 }
12 int main() {
13     cout << area(3,4) << endl;
14     cout << area(5) << endl;
15     cout << area(5,5,6) << endl;
16 }
```

Output:-

```
C:\Users\asus\Desktop\oops\  X  +  v

6
10.825
12

-----
Process exited after 0.2611 seconds with return value 0
Press any key to continue . . .
```

4. Write a program with Student as abstract class and create derive classes Engineering, Medicine and Science from base class Student. Create the objects of the derived classes and process them and access them using array of pointer of type base class Student.

```
ques1.cpp  ques2.cpp  [*] ques3.cpp  ques4.cpp
1  //Assignment- 7
2  //Question -4
3  #include <iostream>
4  using namespace std;
5  class Student {
6  public:
7      virtual void show() = 0;
8  };
9  class Engineering : public Student {
10 public:
11     void show() { cout << "Engineering Student\n"; }
12 };
13 class Medicine : public Student {
14 public:
15     void show() { cout << "Medicine Student\n"; }
16 };
17 class Science : public Student {
18 public:
19     void show() { cout << "Science Student\n"; }
20 };
21 int main() {
22     Student *arr[3];
23     Engineering e; Medicine m; Science s;
24
25     arr[0] = &e;
26     arr[1] = &m;
27     arr[2] = &s;
28
29     for(int i=0;i<3;i++)
30         arr[i]->show();
31 }
```

Output:-

```
C:\Users\asus\Desktop\oops\  x  +  v
Engineering Student
Medicine Student
Science Student

-----
Process exited after 0.3031 seconds with return value 0
Press any key to continue . . .
```

5. Create a class Time with three private variables int h,m,s; Create a function to overload '+' operator to add two time variables.

```
int main(){  
    Time t1(5,15,34),t2(9,53,58),t3;  
    t3 = t1 + t2;  
    t3.show();  
}
```

Code:-

```
1 //Assignment- 7  
2 //Question -5  
3 #include <iostream>  
4 using namespace std;  
5 class Time {  
6     int h, m, s;  
7     public:  
8     Time(int hh=0, int mm=0, int ss=0) {  
9         h=hh; m=mm; s=ss;  
10    }  
11    Time operator+(Time t) {  
12        Time temp;  
13        temp.s = s + t.s;  
14        temp.m = m + t.m + temp.s/60;  
15        temp.s %= 60;  
16        temp.h = h + t.h + temp.m/60;  
17        temp.m %= 60;  
18        return temp;  
19    }  
20    void show() {  
21        cout << h << ":" << m << ":" << s;  
22    }  
23 };  
24 int main() {  
25     Time t1(5,15,34), t2(9,53,58), t3;  
26     t3 = t1 + t2;  
27     t3.show();  
28 }
```

Output:-

```
C:\Users\asus\Desktop\oops\  ×  +  v  
15:9:32  
-----  
Process exited after 0.3129 seconds with return value 0  
Press any key to continue . . .
```

6. Define a class STRING and overload == to compare two strings and + operator for concatenation of two strings.

Code:-

```
Untitled1 ques6.cpp
1 //Assignment- 7
2 //Question -6
3 #include <iostream>
4 #include <cstring>
5 using namespace std;
6 class STRING {
7     char str[100];
8 public:
9     STRING(char s[] = "") {
10         strcpy(str, s);
11     }
12     STRING operator+(STRING s) {
13         STRING temp;
14         strcpy(temp.str, str);
15         strcat(temp.str, s.str);
16         return temp;
17     }
18     void display() {
19         cout << str << endl;
20     }
21 };
22 int main() {
23     STRING s1("Hello ");
24     STRING s2("World");
25
26     STRING s3 = s1 + s2;
27
28     s3.display();
29
30     return 0;
31 }
```

Output:-

```
C:\Users\asus\Desktop\oops\  x  +  v
Hello World
-----
Process exited after 0.2827 seconds with return value 0
Press any key to continue . . .
```

7. Write a program in C++ to create a class matrix and overload * operator using friend function to multiply two matrices.

Code:-

```

Untitled1 ques7.cpp
1 //Assignment- 7
2 //Question -7
3 #include <iostream>
4 using namespace std;
5 class Matrix {
6     int a[2][2];
7 public:
8     void input() {
9         for(int i=0;i<2;i++)
10             for(int j=0;j<2;j++)
11                 cin >> a[i][j];
12     }
13     friend Matrix operator*(Matrix m1, Matrix m2) {
14         Matrix temp;
15         for(int i=0;i<2;i++) {
16             for(int j=0;j<2;j++) {
17                 temp.a[i][j] = 0;
18                 for(int k=0;k<2;k++) {
19                     temp.a[i][j] += m1.a[i][k] * m2.a[k][j];
20                 }
21             }
22         }
23         return temp;
24     }
25     void display() {
26         for(int i=0;i<2;i++) {
27             for(int j=0;j<2;j++)
28                 cout << a[i][j] << " ";
29             cout << endl;
30         }
31     }
32 };
33 int main() {
34     Matrix m1, m2, m3;
35     cout << "Enter first matrix:\n";
36     m1.input();
37     cout << "Enter second matrix:\n";

```

```

38 };
39 int main() {
40     Matrix m1, m2, m3;
41     cout << "Enter first matrix:\n";
42     m1.input();
43     cout << "Enter second matrix:\n";
44     m2.input();
45     m3 = m1 * m2;
46     cout << "Result matrix:\n";
47     m3.display();
48 }

```

Output:-

```

Enter first matrix:
2 3 45 6
Enter second matrix:
4 5 6 35
Result matrix:
26 115
216 435

-----
Process exited after 8.019 seconds with return value 0
Press any key to continue . . .

```

8. Overload '[]' to check array index out of bounds problem at run time.

Code:-

```
ques9.cpp  ques7.cpp  ques8.cpp
1  //Assignment- 7
2  //Question -8
3  #include <iostream>
4  using namespace std;
5  class Array {
6      int arr[5];
7  public:
8      int& operator[](int i) {
9          if(i < 0 || i >= 5) {
10             cout << "Error: Index out of bounds\n";
11             exit(1); // better exit code
12         }
13         return arr[i];
14     }
15 };
16 int main() {
17     Array a;
18     a[0] = 10;
19     a[1] = 20;
20     cout << a[0] << " " << a[1] << endl;
21     // Uncomment to test error
22     // cout << a[10];
23 }
```

Output:-

```
C:\Users\asus\Desktop\oops\  X  +  v
10 20
-----
Process exited after 0.2782 seconds with return value 0
Press any key to continue . . .
```

9. Overload '()' to input arbitrary number of input arguments for an object.

Code:-


```

ques9.cpp ques7.cpp ques8.cpp
1 //Assignment- 7
2 //Question -9
3 #include <iostream>
4 using namespace std;
5
6 class Demo {
7 public:
8     void operator()(int a, int b) {
9         cout << "Sum: " << a + b << endl;
10    }
11 };
12
13 int main() {
14     Demo d;
15
16     d(5, 7); // calls operator()
17
18     return 0;
19 }

```

Output:-

```

C:\Users\asus\Desktop\oops\ x + v
Sum: 12
-----
Process exited after 0.3196 seconds with return value 0
Press any key to continue . . .

```

10. Write a program in C++ to overload input operator (>>) and output operator (<<).

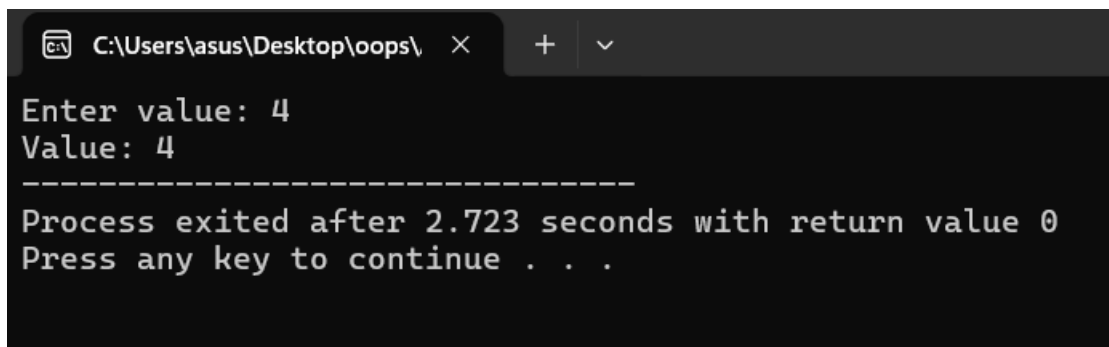
Code:-

```

ques9.cpp ques7.cpp ques8.cpp ques10.cpp
1 //Assignment- 7
2 //Question -10
3 #include <iostream>
4 using namespace std;
5 class Test {
6     int x;
7 public:
8     friend istream& operator>>(istream &in, Test &t) {
9         cout << "Enter value: ";
10        in >> t.x;
11        return in;
12    }
13    friend ostream& operator<<(ostream &out, Test &t) {
14        out << "Value: " << t.x;
15        return out;
16    }
17 };
18 int main() {
19     Test t;
20     cin >> t;
21     cout << t;
22     return 0;
23 }

```

Output:-

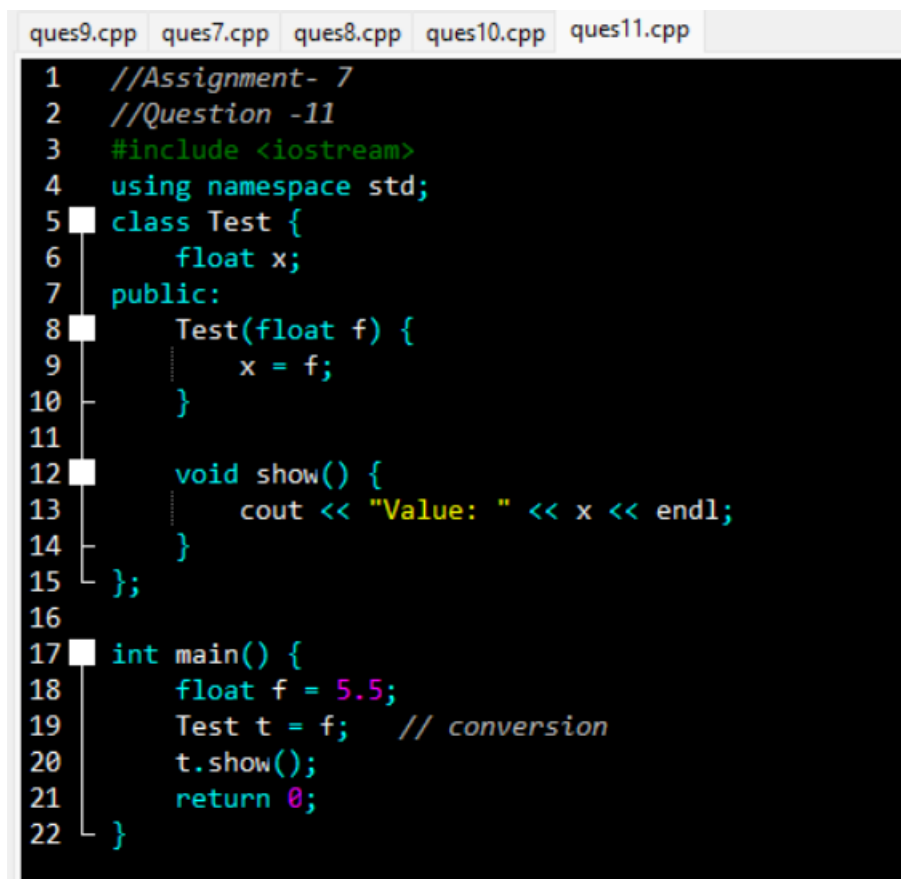


```
C:\Users\asus\Desktop\oops\ × + ∨  
Enter value: 4  
Value: 4  
-----  
Process exited after 2.723 seconds with return value 0  
Press any key to continue . . .
```

11. Write a program to convert basic data type (float) to user defined data type (object).

```
class Test {  
  
private:  
    //...  
  
public:  
    Test(data_type) {  
        // conversion code  
    }  
};
```

Code:-



```
ques9.cpp ques7.cpp ques8.cpp ques10.cpp ques11.cpp  
1 //Assignment- 7  
2 //Question -11  
3 #include <iostream>  
4 using namespace std;  
5 class Test {  
6     float x;  
7     public:  
8     Test(float f) {  
9         x = f;  
10    }  
11  
12    void show() {  
13        cout << "Value: " << x << endl;  
14    }  
15 };  
16  
17 int main() {  
18     float f = 5.5;  
19     Test t = f; // conversion  
20     t.show();  
21     return 0;  
22 }
```

Output:-

```
C:\Users\asus\Desktop\oops\  ×  +  v
Value: 5.5
-----
Process exited after 0.2861 seconds with return value 0
Press any key to continue . . .
```

12. Write a program to convert UDT to basic data type (float)

```
class Test {
public:
    operator data_type() {
        // conversion code
    }
};
```

Code:-

```
ques9.cpp  ques7.cpp  ques8.cpp  ques10.cpp  ques11.cpp  ques12.cpp  ques13.cpp
1  //Assignment- 7
2  //Question -12
3  #include <iostream>
4  using namespace std;
5  class Test {
6      float x;
7  public:
8      Test(float f) {
9          x = f;
10     }
11     operator float() {
12         return x;
13     }
14 };
15 int main() {
16     Test t(7.5);
17     float f;
18     f = t; // conversion
19     cout << "Converted value: " << f << endl;
20     return 0;
21 }
22
23
```

Output:-

```
C:\Users\asus\Desktop\oops\  X + v
Converted value: 7.5
-----
Process exited after 0.2919 seconds with return value 0
Press any key to continue . . .
```

13. How will you convert one UDT to another UDT. For example conversion of polar to cartesian system.

Polar p(10,5);

Cartesian c = p;

c.show();

Code:-

```
ques9.cpp ques7.cpp ques8.cpp ques10.cpp ques11.cpp ques12.cpp ques13.cpp
1 //Assignment- 7
2 //Question -13
3 #include <iostream>
4 #include <cmath>
5 using namespace std;
6 class Cartesian;
7 class Polar {
8     float r, theta;
9 public:
10     Polar(float r1, float t1) {
11         r = r1;
12         theta = t1;
13     }
14     operator Cartesian();
15 };
16 class Cartesian {
17     float x, y;
18 public:
19     Cartesian(float a=0, float b=0) {
20         x = a;
21         y = b;
22     }
23     void show() {
24         cout << "x = " << x << " y = " << y << endl;
25     }
26 };
27 Polar::operator Cartesian() {
28     return Cartesian(r * cos(theta), r * sin(theta));
29 }
30 int main() {
31     Polar p(10, 0.785); // ~45 degrees
32     Cartesian c;
33     c = p; // conversion
34     c.show();
35 }
```

Output:-

```
C:\Users\asus\Desktop\oops\ × + v
x = 7.07388 y = 7.06825
-----
Process exited after 0.2541 seconds with return value 0
Press any key to continue . . .
```